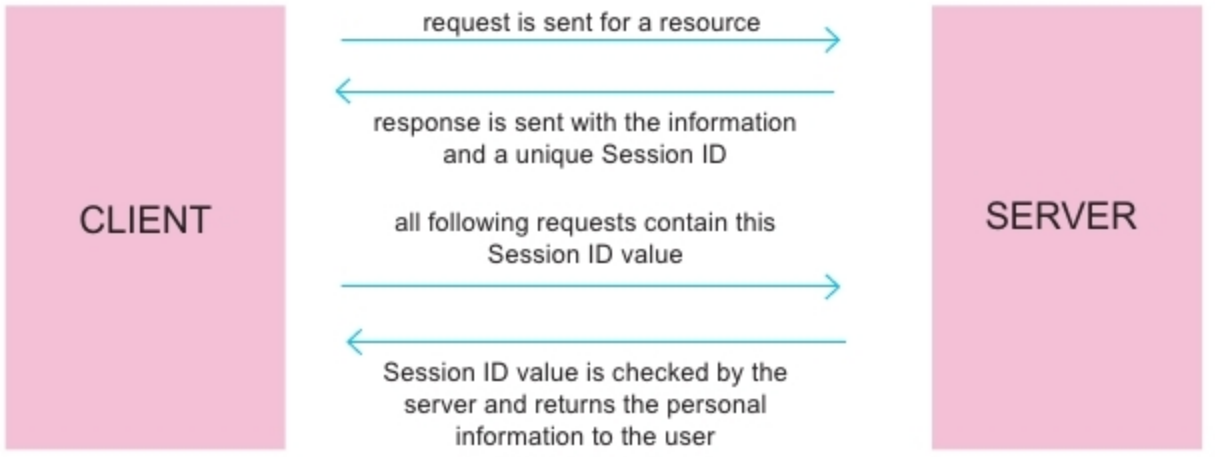

Figure 2: Modified protocol with session management

is globally known as a session.

State management flow with sessions and cookies

Now that we are using a state-management tool, let's look at the flow of data when a client tries to log in using authentic credentials from the server:

1. The client visits the login page of a website and fills out the login form using the user name, email and password.
2. After submitting the form, the credentials are validated at the server end and the request is authenticated.
3. If the entered credentials are valid, the server generates a unique random number, known as the session ID, which is also stored on the server in a specific folder in which other session-specific information is stored.
4. The session ID is sent back to the user in the cookie header of the response data. For Node.js, this cookie header is named *connect.sid*.
5. On the client side, if cookies are enabled by the user on the browser, they are saved in the memory space set aside by the user exclusively for cookies.
6. For each request that goes to the Web server from this client, the *connect.sid* cookie is passed back in the header file. On coming across this, the server tries to initialise a session with that particular session ID. This is done by loading the session file that was created earlier when the session was being initialised. Then, a global array variable (*request* in case of Node.js) is initialised with the data stored in this session file.

Thus, this authentication data is preserved with all sequential incoming requests and the user is kept logged in throughout the session until he or she manually logs off.

Let's look at a working example. Now that we've seen how important it is to have sessions, let's build a mini Web application using Node.js, with Express as a framework and a few other modules from the Node Package Manager.

Note: This article assumes that the reader has basic Node.js and jQuery knowledge, apart from also having worked with npm on the command line and locally hosted a Node.js app.

To demonstrate session management, we will use a basic log-in/log-out system where the user logs in using a user name, email and password and the session is linked to the user via the provided user name and email. Upon logging out, the session is destroyed and the user will be redirected to the home page, where he or she can log in again.

Setting up the Node.js project

Create a new folder and open a terminal in that directory.

Note: Here, we are using the Windows command line tool.

```
npm init --y
```

This creates the *package.json* file for building the Node project. Let's now install all the required dependencies:

```
npm install --save express body-parser
express-session
```

Now that the dependencies are installed, let us start coding the main app with routes logic.

We will use a module called express-session, which acts as the middleware for internally handling our sessions. This also requires that we use Express in our project.

```
const express = require('express');
const session = require('express-
session');
const bodyParser = require('body-
parser');
```

We initialise the session as follows:

```
app.use(
  session({
    secret: 'thisisasecret',
    saveUninitialized: false,
    resave: false
  })
);
```

Here, the *secret* is the hash key passed to the header file so that this information can be securely decoded at the server end only. This is important because without a session *secret*, any third party app can easily look into the cookie and hijack it. Having a *secret* ensures that when authenticating, the cookie data is encoded using the *secret* key only known to the client and the server; this data can then not be easily decoded by a third listener.

Using the *request* variable, the session can be assigned to any variable